DA5300: Data Structures For Data Science

Laboratory 9 - Shortest Paths in Graphs

1 Objectives

In this lab, you will implement and compare three classical shortest-path algorithms:

1. Dijkstra's algorithm.
2. Bellman–Ford algorithm.
3. Floyd–Warshall algorithm.

You will represent a graph using an edge list, construct a weighted adjacency matrix when required, test the algorithms on different graphs, and compare their correctness and computational complexity.

2 Graph Input Format

A weighted directed graph is represented by:

- V: the number of vertices, numbered from 0 to $V - 1$.
- `edges`: a list of triples (u, v, w), where u is the source vertex, v is the destination vertex, and w is the edge weight.

For example, the graph with edges $0 \to 1$ of weight 4 and $0 \to 2$ of weight 7 is represented by

```
V = 3
edges = [
    (0, 1, 4),
    (0, 2, 7)
]
```

The graph is directed: an edge (u, v, w) does not imply an edge (v, u, w).

Use `float('inf')` to represent the absence of an edge in an adjacency matrix. The diagonal entries of the matrix are zero.

2.1 Provided helper function

Use the following helper function to construct the weighted adjacency matrix. You do not need to implement this function.

```
def make_adjacency_matrix(V, edges):
    INF = float('inf')
    W = [[INF] * V for _ in range(V)]

    for i in range(V):
```

```
        W[i][i] = 0

    for u, v, w in edges:
        W[u][v] = w

    return W
```

For algorithms that use an adjacency matrix, construct it as follows:

```
W = make_adjacency_matrix(V, edges)
```

The function inputs for the three algorithms are:

Algorithm	Input
Dijkstra	`W, source`
Bellman–Ford	`V, edges, source`
Floyd–Warshall	`W`

3 Part A: Dijkstra's Algorithm

3.1 Problem

Implement Dijkstra's algorithm to compute the shortest distances from a source vertex to all other vertices in a weighted directed graph with non-negative edge weights.

3.2 Test graph

Use the following input:

```
V = 5
edges = [
    (0, 1, 4),
    (0, 3, 1),
    (3, 1, 2),
    (1, 2, 1),
    (2, 4, 3),
    (3, 4, 2)
]
source = 0
```

3.3 Tasks

(a) Construct the adjacency matrix using the provided helper.

(b) Implement Dijkstra's algorithm.

(c) Return the shortest distance from the source to every vertex.

(d) Explain why Dijkstra's algorithm cannot, in general, handle negative edge weights correctly.

3.4 Function template

```python
def dijkstra(W, source):
    """
    W: weighted adjacency matrix
    source: source vertex

    Return a list dist, where dist[v] is the shortest
    distance from source to vertex v.
    """
    V = len(W)

    # TODO: Initialise the distance array.
    # TODO: Repeatedly select the unprocessed vertex
    #       with the smallest tentative distance.
    # TODO: Relax its outgoing edges.
    # TODO: Return the distance array.

    pass
```

3.5 Expected result

The shortest distances from vertex 0 are

$$\boxed{[0,\ 3,\ 4,\ 1,\ 3]}.$$

4 Part B: Bellman–Ford Algorithm

4.1 Problem

Implement Bellman–Ford to compute shortest distances from a source vertex. The algorithm supports negative edge weights and can detect a negative weight cycle reachable from the source.

4.2 Test graph

Use the following input:

```
V = 5
edges = [
    (0, 1,  6),
    (0, 2,  7),
    (1, 2,  8),
    (1, 3,  5),
    (1, 4, -4),
    (2, 3, -3),
    (2, 4,  9),
    (3, 1, -2),
    (4, 3,  7)
]
source = 0
```

4.3 Tasks

(a) Implement Bellman–Ford using the edge list directly.

(b) Relax every edge for at most $V - 1$ passes.

(c) Stop early if a complete pass makes no changes.

(d) Perform an additional pass to detect a reachable negative weight cycle.

(e) Return the distance array and an indicator of whether such a cycle exists.

4.4 Function template

```python
def bellman_ford(V, edges, source):
    """
    V: number of vertices
    edges: list of (u, v, weight) tuples
    source: source vertex

    Return (dist, has_negative_cycle).
    """
    INF = float('inf')
    dist = [INF] * V
    dist[source] = 0

    # TODO: Relax every edge V - 1 times.
    # TODO: Optionally stop early if no distance changes.
    # TODO: Check for a reachable negative weight cycle.

    pass
```

4.5 Expected result

For this graph, the shortest distances from vertex 0 are

$$\boxed{[0,\ 2,\ 7,\ 4,\ -2]}.$$

There is no reachable negative weight cycle.

5 Part C: Floyd–Warshall Algorithm

5.1 Problem

Implement Floyd–Warshall to compute shortest distances between every pair of vertices.

The algorithm considers each vertex in turn as a possible intermediate vertex on a shortest path.

5.2 Tasks

(a) Construct the adjacency matrix for the graph in Part B.

(b) Initialise the distance matrix using the adjacency matrix.

(c) Implement the three nested loops of Floyd–Warshall.

(d) Return the all-pairs shortest-distance matrix.

(e) Explain how a negative diagonal entry can reveal a negative weight cycle.

5.3 Function template

```python
def floyd_warshall(W):
    """
    W: weighted adjacency matrix

    Return the all-pairs shortest-distance matrix.
    """
    V = len(W)
    dist = [row[:] for row in W]

    # TODO: Consider each vertex k as an intermediate vertex.
    # TODO: Update dist[i][j] using dist[i][k] + dist[k][j].
    # TODO: Return the final distance matrix.

    pass
```

5.4 Test graph and expected result

Use the graph from Part B and construct its adjacency matrix using `make_adjacency_matrix`.

The expected final distance matrix is

$$D = \begin{bmatrix} 0 & 2 & 7 & 4 & -2 \\ \infty & 0 & \infty & 5 & -4 \\ \infty & \infty & 0 & -3 & \infty \\ \infty & -2 & \infty & 0 & -6 \\ \infty & \infty & \infty & 7 & 0 \end{bmatrix}.$$

A negative diagonal entry indicates a negative weight cycle. If no negative cycle exists, the diagonal entries remain zero.

6 Part D: Negative Cycle Experiment

Modify the graph from Part B by adding the edge

$$(4, 1, -1).$$

(a) Run Bellman–Ford from vertex 0.

(b) Determine whether a reachable negative weight cycle exists.

(c) Run Floyd–Warshall on the modified graph.

(d) Check whether any diagonal entry becomes negative.

(e) Explain why a reachable negative cycle makes shortest distances to affected vertices unbounded below.

7 Part E: Running the Algorithms

Use the following calls to test your implementations.

```python
# Part A
V = 5
edges = [
    (0, 1, 4), (0, 3, 1), (3, 1, 2),
    (1, 2, 1), (2, 4, 3), (3, 4, 2)
]
W = make_adjacency_matrix(V, edges)
print(dijkstra(W, 0))

# Part B
V = 5
edges = [
    (0, 1, 6), (0, 2, 7), (1, 2, 8),
    (1, 3, 5), (1, 4, -4), (2, 3, -3),
    (2, 4, 9), (3, 1, -2), (4, 3, 7)
]
W = make_adjacency_matrix(V, edges)
print(bellman_ford(V, edges, 0))
print(floyd_warshall(W))
```